

1. Introduction

Linux is one of the most important operating systems in the field of cybersecurity. It is widely used by security professionals because it provides powerful tools for managing computers, networks, users, files, and security services.

Unlike a graphical operating system where most activities are performed by clicking buttons and menus, Linux allows users to perform many activities directly from the command line. The command line gives a user direct control over the operating system through commands.

Understanding Linux is therefore important for a cybersecurity student because many security tools and servers operate on Linux systems. Security professionals also use Linux commands to investigate systems, manage accounts, control permissions, monitor services, and perform security testing.

This task focused on three important areas of Linux security:

1. File permissions
2. User management
3. Secure Shell (SSH)

The practical work was performed using two virtual machines in VMware Workstation. Kali Linux, named Grazy Cat, was used as the main cybersecurity workstation. Basic Pentesting 1, which contains an Ubuntu-based system, was used as the remote Linux server.

The main practical activity was to establish a secure SSH connection from Kali Linux to the Ubuntu system.

2. Objectives

The main objective of this task was to develop basic Linux command-line and security skills.

The specific objectives were to:

- Understand the Linux command-line environment.
- Learn basic Linux commands.
- Learn how files and directories are managed from the terminal.
- Understand Linux file permissions.
- Understand the meaning of read, write, and execute permissions.
- Learn how to change file permissions.
- Understand Linux users and groups.
- Learn basic user-management commands.
- Understand the purpose of SSH.
- Configure and check an SSH service.
- Connect to an Ubuntu system remotely using Kali Linux.
- Understand how SSH can be made more secure.
- Apply Linux security concepts in a controlled virtual laboratory.

3. Practical Laboratory Environment

The practical was performed in a controlled virtual environment using VMware Workstation.

Two Linux virtual machines were used.

3.1 Kali Linux

The first machine was my Kali Linux virtual machine.

Operating System: Kali Linux

Role: Main cybersecurity workstation and SSH client

Kali Linux was used as the main machine because it provides a Linux command-line environment and is designed for cybersecurity learning and security testing.

The majority of the practical commands were performed from Kali Linux.

3.2 Basic Pentesting 1

The second machine was Basic Pentesting 1.

VM Name: Basic Pentesting 1

Operating System: Ubuntu

Role: Remote SSH server

This machine was selected because the original Ubuntu installation had problems. Instead of installing another Ubuntu system, an existing Ubuntu-based virtual machine, Basic Pentesting 1, was used for the practical.

3.3 Network Environment

Both machines were connected to the same VMware virtual network using the 192.168.94.x address range.

The basic communication model was:

VMware Virtual Network

This setup allowed Kali Linux to communicate with the Ubuntu system without using an external computer.

All practical activities were performed within this controlled laboratory environment.

4. Understanding the Linux Command Line

The Linux command line is a text-based interface used to communicate directly with the operating system.

Instead of clicking on folders and applications, a user types commands into a terminal.

For example, the command:

- ***pwd***

asks Linux to display the current directory.

The command line is important in cybersecurity because many security tasks require direct interaction with the operating system.

Examples include:

- Checking users.
- Checking file permissions.
- Managing files.
- Checking network information.
- Starting and stopping services.
- Checking security logs.
- Connecting to remote systems.
- Troubleshooting security problems.

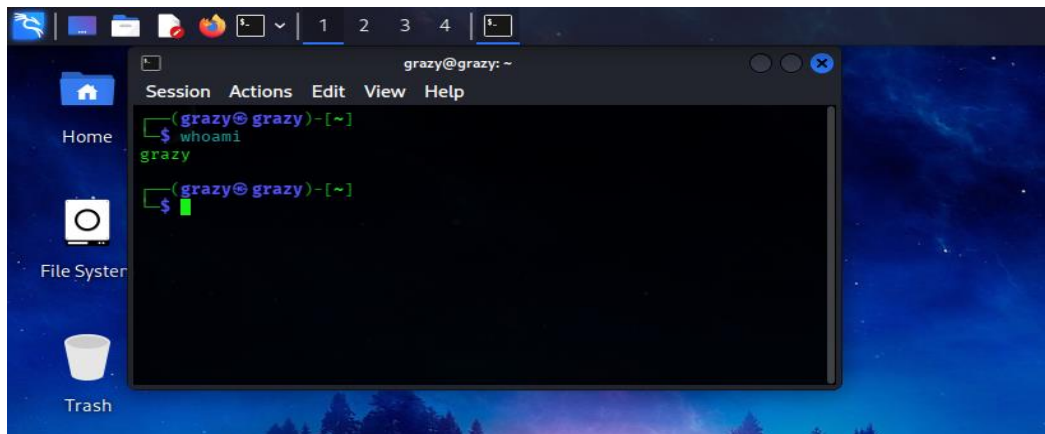
5. Basic Linux Commands

Several basic commands were studied and practiced.

5.1 whoami

The **whoami** command displays the username of the person currently logged into the Linux system.

Whoami



For example, the result may be: kali, root or another username depending on the account being used.

Security importance

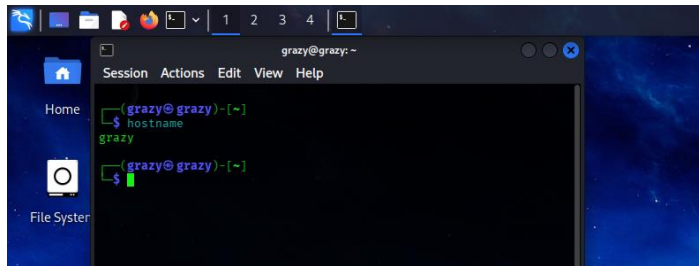
This command is useful because it confirms which account is currently being used.

This is particularly important when working with multiple users or when using administrative privileges. A security professional should always know which account is executing commands.

6. hostname

The **hostname** command displays the name of the computer.

hostname



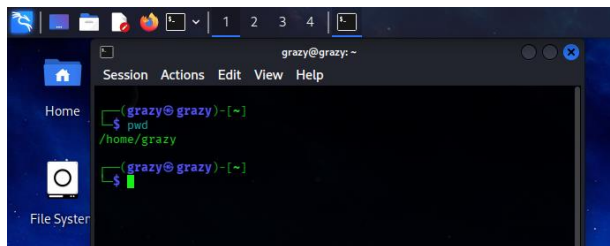
This is useful when working with several virtual machines because it helps identify which system is currently being accessed.

For example, when connected through SSH, hostname can help confirm that the user is working on the remote Ubuntu machine rather than the local Kali machine.

7. pwd

The **pwd** command means Print Working Directory.

Pwd



It shows the current location within the Linux file system.

For example:

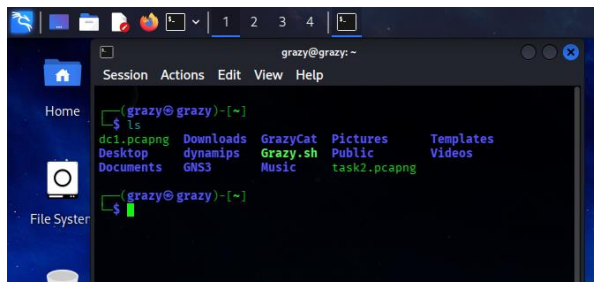
/home/kali

Knowing the current directory is important because many commands operate on files relative to the current location.

8. ls

The **ls** command displays files and directories.

ls



A more detailed version is:

ls -l

The -l option provides additional information such as:

- File permissions
- File owner
- Group
- File size
- Date/time
- File name

Hidden files can be displayed using:

ls -la

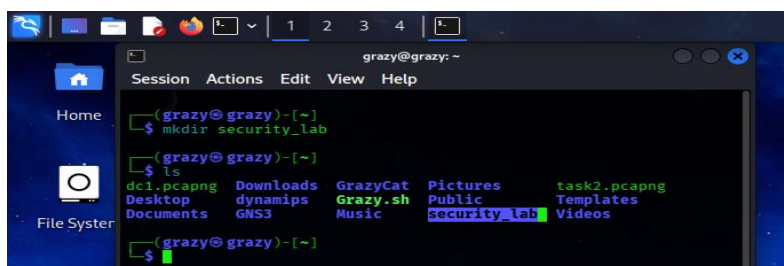
This command is particularly important for security because file permissions and ownership can be viewed using ls -l.

9. Creating and Managing Files

Linux allows users to create and manage files directly from the terminal.

A directory can be created using:

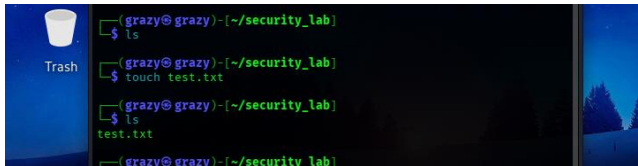
`mkdir security_lab`



The mkdir command means make directory.

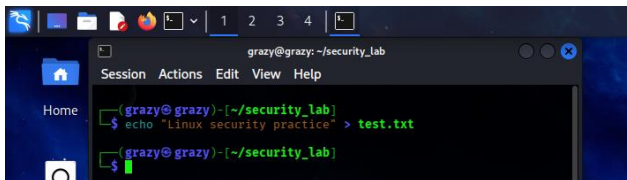
A file can be created using:

`touch test.txt`



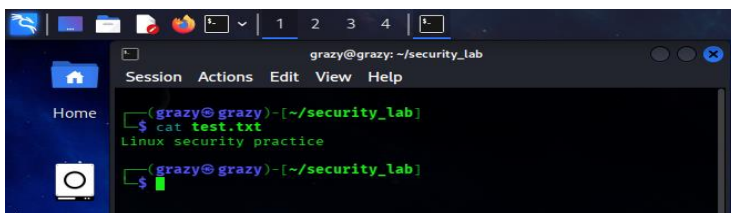
Information can then be written into the file:

```
echo "Linux security practice" > test.txt
```



The file contents can be displayed using:

```
cat test.txt
```



These commands demonstrate how Linux users can manage files without using a graphical file manager.

10. File Permissions

One of the most important Linux security concepts is file permissions.

Linux is designed as a multi-user operating system. This means that many different users can exist on the same computer.

Because different users may have different levels of authority, Linux needs a mechanism to determine who can access a particular file.

This is where file permissions are used.

File permissions determine whether a user can:

- Read a file.
- Modify a file.
- Execute a file.

The three basic permissions are:

Permission	Symbol	SymbolMeaning
Read	r	Allows the contents of a file to be viewed

Write	w	Allows the file to be modified
Execute	x	Allows a file to be executed

11. Understanding Linux Permission Groups

Linux permissions are normally associated with three categories:

- **Owner**

The owner is the user who owns the file.

- **Group**

The group represents users who belong to the file's assigned group.

- **Others**

Others refers to users who are not the owner and are not members of the assigned group.

For example:

-rwxr-xr--

The permissions can be understood as:

- Owner → rwx
- Group → r-x
- Others → r--

This means:

- The owner can read, write, and execute.
- The group can read and execute.
- Other users can only read.

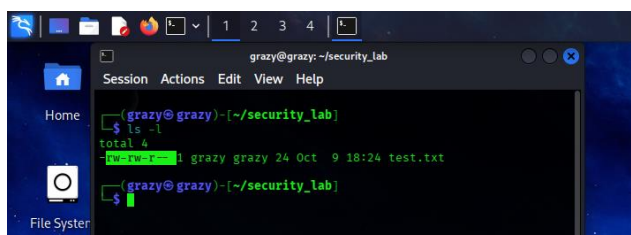
Understanding these permissions is important because incorrectly configured permissions can expose sensitive information or allow unauthorized modification.

12. Viewing File Permissions

File permissions can be viewed using:

ls -l

For example:



```

grazy@grazy:~/security_lab
Session Actions Edit View Help
$ ls -l
total 4
-rwxr-xr-- 1 grazy grazy 24 Oct 9 18:24 test.txt
$

```

The permission section appears at the beginning of the line.

The **ls -l** command therefore provides useful security information about files.

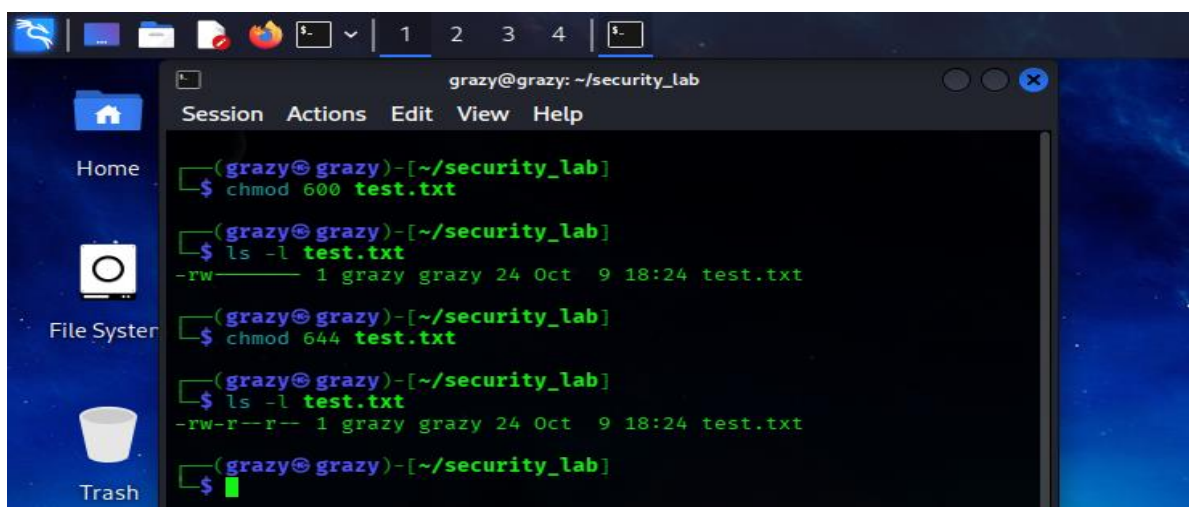
It allows an administrator or security professional to determine:

- Who owns the file.
- Which group owns it.
- Who can read it.
- Who can modify it.
- Who can execute it.

13. Changing File Permissions with chmod

The **chmod** command is used to modify file permissions.

For example:

A screenshot of a terminal window titled 'grazy@grazy: ~/security_lab'. The window shows a series of commands and their outputs. First, the user runs 'chmod 600 test.txt'. Then, they run 'ls -l test.txt', which shows the file permissions as '-rw----- 1 grazy grazy 24 Oct 9 18:24 test.txt'. Next, the user runs 'chmod 644 test.txt'. Finally, they run 'ls -l test.txt' again, showing the updated permissions as '-rw-r--r-- 1 grazy grazy 24 Oct 9 18:24 test.txt'. The terminal window has a dark background with green and white text. On the left side of the terminal, there is a sidebar with icons for Home, File System, and Trash.

The permission value 600 means that the owner receives read and write access while access for group members and other users is removed.

The result can be checked using: **ls -l test.txt**

Another example is: **chmod 644 test.txt**

This allows the owner to read and write the file, while other users can read it.

14. Why File Permissions Matter in Cybersecurity

File permissions are an important security control.

Imagine a file containing:

- Password information
- Configuration information
- Private keys
- Personal information
- Security logs

If the permissions are too open, unauthorized users may be able to read or modify the information.

For example, giving every user write access to an important system file could allow an unauthorized user to change the file.

Therefore, file permissions should follow the principle of least privilege.

The principle of least privilege means that users should receive only the permissions they actually need.

15. Linux User Management

Linux supports multiple users.

Each user can have their own:

- Username
- Password
- User ID
- Group membership
- Permissions
- Access rights

User management is important because cybersecurity depends heavily on controlling who can access a system.

A system administrator should know which accounts exist and what privileges those accounts have.

16. Checking the Current User

The following command identifies the current user:

- **whoami**

More detailed information can be obtained using:

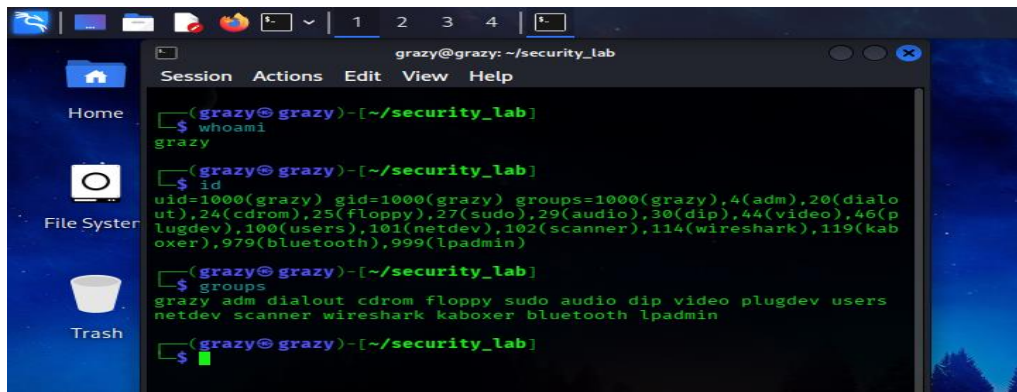
- **id**

The **id** command can show information such as:

- User ID
- Group ID
- Groups associated with the user

Group membership can also be checked using:

- Groups

A terminal window titled 'grazy@grazy: ~/security_lab' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows three commands and their outputs: 1. '\$ whoami' outputs 'grazy'. 2. '\$ id' outputs 'uid=1000(grazy) gid=1000(grazy) groups=1000(grazy),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(scanner),114(wireshark),119(kaboxer),979(bluetooth),999(lpadmin)'. 3. '\$ groups' outputs 'grazy adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner wireshark kaboxer bluetooth lpadmin'.

These commands help security administrators understand the privileges associated with an account.

17. Creating a User

A new user can be created using:

sudo adduser securityuser

The system will request information such as a password.

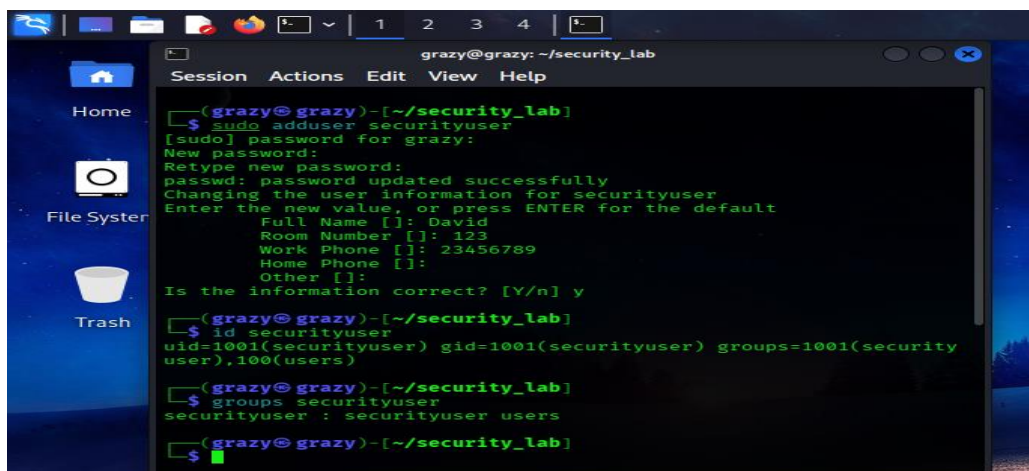
After creating the account, it can be checked using:

- `id securityuser`

Group membership can be checked using:

- `groups securityuser`

A dedicated account is useful for security testing because it allows the activity to be performed without unnecessarily using a highly privileged account.

A terminal window titled 'grazy@grazy: ~/security_lab' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the execution of 'sudo adduser securityuser'. It prompts for a password, which is entered twice. Then it prompts for user information: Full Name (David), Room Number (123), Work Phone (23456789), Home Phone, and Other. The user confirms the information is correct with 'y'. Finally, it shows the output of 'id securityuser' (uid=1001(securityuser) gid=1001(securityuser) groups=1001(securityuser),100(users)) and 'groups securityuser' (securityuser : securityuser users).

This demonstrates Linux user and group information.

18. Importance of User Management in Security

Poor user management can create security risks.

For example:

- Unused accounts may remain active.
- Users may have excessive privileges.
- Weak passwords may be used.
- Multiple people may share one account.
- Administrators may use privileged accounts unnecessarily.

Good user management helps reduce these risks.

A secure system should provide users with only the access they require.

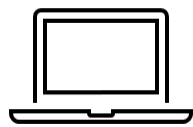
19. Introduction to SSH

SSH stands for Secure Shell.

SSH is a protocol used to securely connect to and manage another computer over a network.

Instead of physically sitting in front of the remote computer or server, an administrator can use SSH from another computer to securely access its command line over the network.

In this practical:



Kali Linux



SSH



Ubuntu / Basic Pentesting 1

Kali acts as the SSH client, while Ubuntu acts as the SSH server.

20. Why SSH is Important in Cybersecurity

SSH is important because system administrators and cybersecurity professionals often need to manage remote Linux servers.

SSH provides a secure method for:

- Remote administration
- Running commands
- Managing files
- Checking services
- Troubleshooting systems
- Managing users
- Monitoring systems

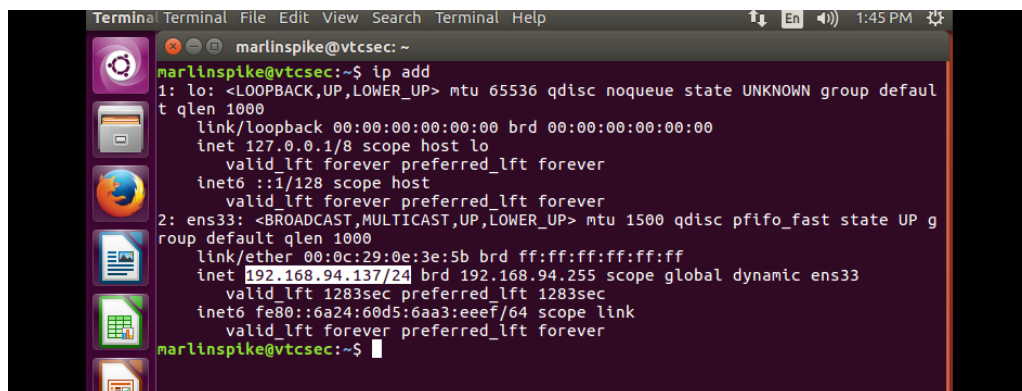
SSH is called Secure Shell because communication between the client and server is protected rather than being sent as ordinary unencrypted terminal traffic.

21. Finding the Ubuntu IP Address

Before Kali can connect to Ubuntu, Kali needs to know the Ubuntu machine's IP address.

On Ubuntu, the following command can be used:

ip addr

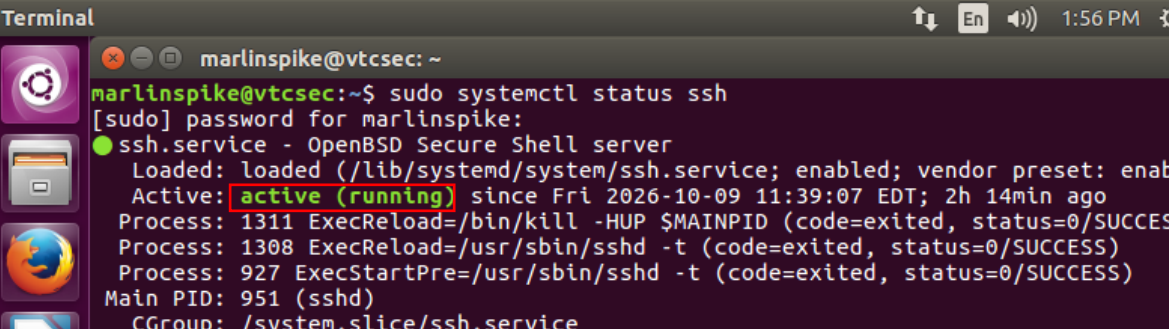
A screenshot of a terminal window titled 'Terminal' with a menu bar (Terminal, File, Edit, View, Search, Terminal, Help) and a status bar (1:45 PM). The terminal shows the user 'marlinspike@vtcsec' at the prompt '~'. The command 'ip addr' has been executed, displaying network interface details for 'lo' and 'ens33'. The 'ens33' interface is highlighted with a yellow box, showing its IP address as '192.168.94.137/24'.

```
marlinspike@vtcsec: ~  
marlinspike@vtcsec:~$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000  
    link/ether 00:0c:29:0e:3e:5b brd ff:ff:ff:ff:ff:ff  
    inet 192.168.94.137/24 brd 192.168.94.255 scope global dynamic ens33  
        valid_lft 1283sec preferred_lft 1283sec  
    inet6 fe80::6a24:60d5:6aa3:eeef/64 scope link  
        valid_lft forever preferred_lft forever  
marlinspike@vtcsec:~$
```

Ubuntu IP Address Used: **192.168.94.137**

22. Checking the SSH Service

Before establishing an SSH connection, the SSH service was checked on the target Linux machine to confirm that the SSH server was operational. The following command was used: **systemctl status ssh**



Terminal

marlinspike@vtcsec: ~

```
marlinspike@vtcsec:~$ sudo systemctl status ssh
[sudo] password for marlinspike:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enab
   Active: active (running) since Fri 2026-10-09 11:39:07 EDT; 2h 14min ago
   Process: 1311 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCE
   Process: 1308 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 927 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 951 (sshd)
   CGroup: /system.slice/ssh.service
           └─951 /usr/sbin/sshd -D

Oct 09 11:39:27 vtcsec systemd[1]: Reloading OpenBSD Secure Shell server.
Oct 09 11:39:27 vtcsec sshd[951]: Received SIGHUP; restarting.
Oct 09 11:39:27 vtcsec systemd[1]: Reloaded OpenBSD Secure Shell server.
Oct 09 11:39:27 vtcsec sshd[951]: Server listening on 0.0.0.0 port 22.
Oct 09 11:39:27 vtcsec sshd[951]: Server listening on :: port 22.
Oct 09 11:39:27 vtcsec systemd[1]: Reloading OpenBSD Secure Shell server.
Oct 09 11:39:37 vtcsec sshd[951]: Received SIGHUP; restarting.
Oct 09 11:39:37 vtcsec systemd[1]: Reloaded OpenBSD Secure Shell server.
Oct 09 11:39:37 vtcsec sshd[951]: Server listening on 0.0.0.0 port 22.
Oct 09 11:39:37 vtcsec sshd[951]: Server listening on :: port 22.
lines 1-20/20 (END)
```

The command output confirmed that the OpenBSD Secure Shell server was loaded and its status was **active (running)**. This indicates that the SSH service was successfully running on the target machine and was ready to accept incoming SSH connection requests.

The output also confirmed that the SSH server was listening on TCP port 22, as shown by the following messages:

- Server listening on 0.0.0.0 port 22
- Server listening on: port 22

This demonstrates that the SSH service was listening for incoming connections on the target machine. The service was also shown as **enabled**, indicating that it was configured to start automatically when the system boots.

Practical Observation

The SSH service was successfully verified as **active and running**, and the server was listening on port 22. Therefore, the target machine was ready for the next stage of the practical exercise: establishing an SSH connection from the Kali Linux client.

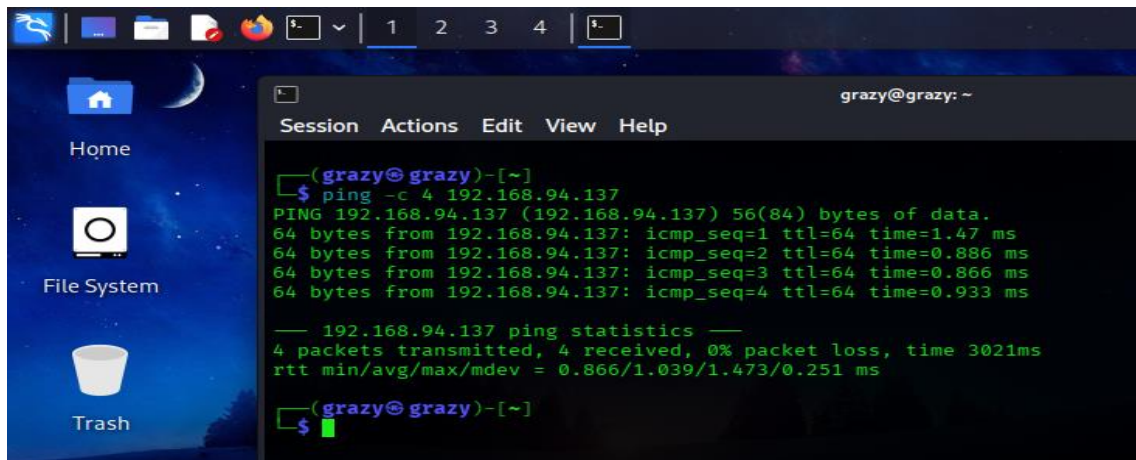
23. Testing Network Connectivity

Before attempting to establish an SSH connection, network connectivity between the Kali Linux client and the target Linux machine was tested. This step was performed to confirm that both virtual machines could communicate with each other through the VMware virtual network.

From the Kali Linux terminal, the ping command can be used to test connectivity to the target machine: `ping -c 4 192.168.94.137`

The IP address 192.168.94.137 represents the target Linux machine identified in the previous section.

The -c 4 option instructs the ping command to send four ICMP echo requests to the target machine. If responses are received, this indicates that the two virtual machines can communicate across the network.

A screenshot of a Kali Linux desktop environment. The desktop background is a dark blue space-themed wallpaper. On the left side, there are icons for 'Home', 'File System', and 'Trash'. A terminal window is open in the center-right, showing a successful ping command. The terminal title bar says 'grazy@grazy: ~'. The terminal output shows four successful ping requests to 192.168.94.137 with varying response times. The terminal also displays ping statistics: 4 packets transmitted, 4 received, 0% packet loss, and a total time of 3021ms.

```
(grazy@grazy)-[~]
$ ping -c 4 192.168.94.137
PING 192.168.94.137 (192.168.94.137) 56(84) bytes of data.
64 bytes from 192.168.94.137: icmp_seq=1 ttl=64 time=1.47 ms
64 bytes from 192.168.94.137: icmp_seq=2 ttl=64 time=0.886 ms
64 bytes from 192.168.94.137: icmp_seq=3 ttl=64 time=0.866 ms
64 bytes from 192.168.94.137: icmp_seq=4 ttl=64 time=0.933 ms

— 192.168.94.137 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3021ms
rtt min/avg/max/mdev = 0.866/1.039/1.473/0.251 ms
(grazy@grazy)-[~]
$
```

The successful exchange of packets confirms that the target machine is reachable from Kali Linux. This is an important troubleshooting step because a failure to establish an SSH connection may be caused by a network connectivity problem rather than an issue with the SSH service itself.

Practical Observation: Network connectivity was tested between Kali Linux and the target machine before proceeding with the SSH connection. The test confirmed that the target machine was reachable through the virtual network.

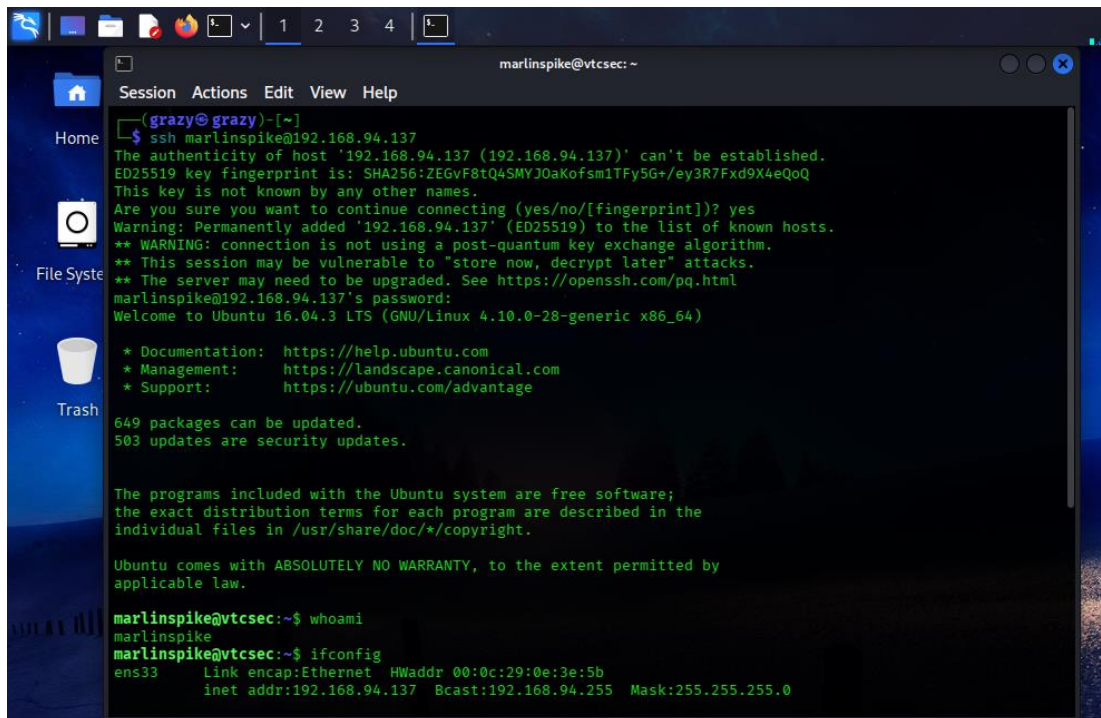
24. Connecting to Ubuntu Through SSH

After confirming that the target system was reachable and that the SSH service was running, an SSH connection was established from Kali Linux to the Basic Pentesting 1 machine.

The following command was used from Kali Linux: **ssh marlinspike@192.168.94.137**

During the first connection, SSH displayed a host authenticity warning and an ED25519 fingerprint. The connection was accepted by entering yes, after which the password for the marlinspike account was entered.

Establishing an SSH connection from Kali Linux to the target Ubuntu system

A screenshot of a Kali Linux desktop environment. The desktop background is a dark blue image. On the left side, there is a sidebar with icons for Home, File System, and Trash. The main window is a terminal titled 'marlinspike@vtcsec: ~'. The terminal shows the following output:

```
(grazy@grazy)-[~]
$ ssh marlinspike@192.168.94.137
The authenticity of host '192.168.94.137 (192.168.94.137)' can't be established.
ED25519 key fingerprint is: SHA256:ZEGvF8tQ4SMYJ0aKofsm1TFy5G+/ey3R7Fxd9X4eQoQ
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.94.137' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
marlinspike@192.168.94.137's password:
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

649 packages can be updated.
503 updates are security updates.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

marlinspike@vtcsec:~$ whoami
marlinspike
marlinspike@vtcsec:~$ ifconfig
ens33    Link encap:Ethernet  HWaddr 00:0c:29:0e:3e:5b
         inet addr:192.168.94.137  Bcast:192.168.94.255  Mask:255.255.255.0
```

The connection was successful, and the terminal displayed the **Ubuntu 16.04.3 LTS** welcome message. The **whoami** command returned marlinspike, confirming that the remote session was using the intended user account. The **ifconfig** command also showed the target machine's IP address as 192.168.94.137.

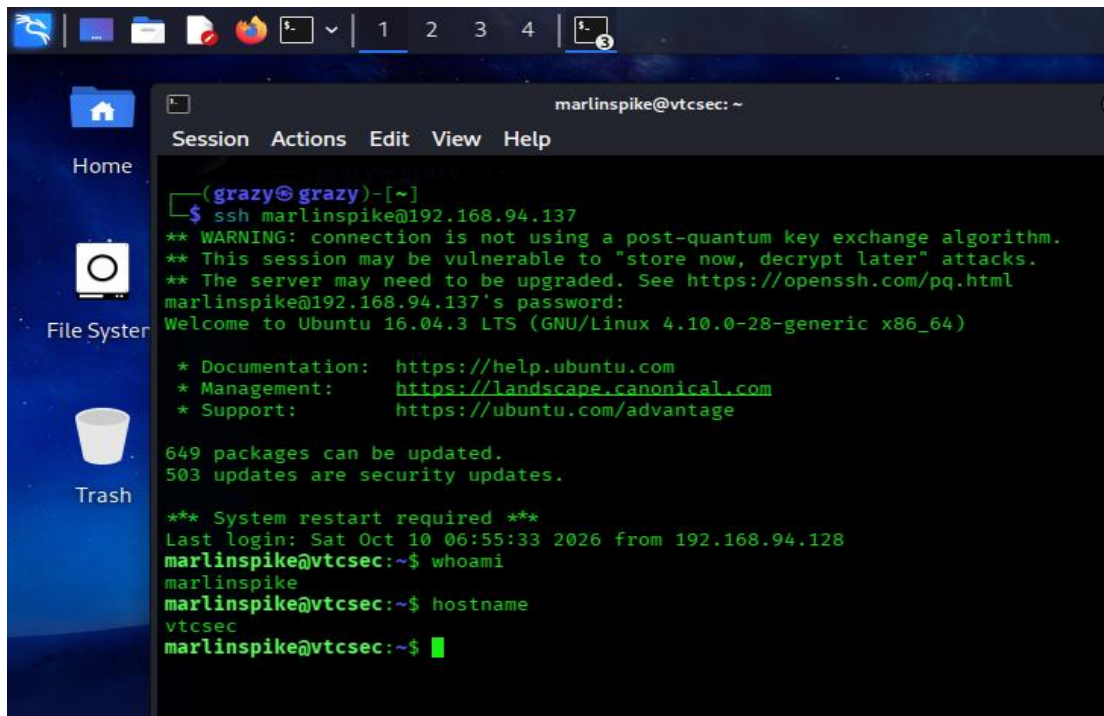
Practical Observation:

The successful SSH connection confirmed that Kali Linux could remotely access the Ubuntu target through SSH using the marlinspike account. This demonstrated the practical use of SSH for secure remote administration.

25. Verifying the Remote Connection

After successfully establishing the SSH connection, the remote session was verified using the **whoami** and **hostname** commands.

Verifying the logged-in user and hostname of the remote Linux system



```
marlinspike@vtcsec: ~  
Session Actions Edit View Help  
(grazy@grazy)-[~]  
$ ssh marlinspike@192.168.94.137  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
marlinspike@192.168.94.137's password:  
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-28-generic x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/advantage  
  
649 packages can be updated.  
503 updates are security updates.  
  
*** System restart required ***  
Last login: Sat Oct 10 06:55:33 2026 from 192.168.94.128  
marlinspike@vtcsec:~$ whoami  
marlinspike  
marlinspike@vtcsec:~$ hostname  
vtcsec  
marlinspike@vtcsec:~$
```

The `whoami` command is used to identify the currently logged-in user, while the `hostname` command displays the name of the computer. In this practical exercise, the expected results are `marlinspike` for the user account and `vtcsec` for the hostname.

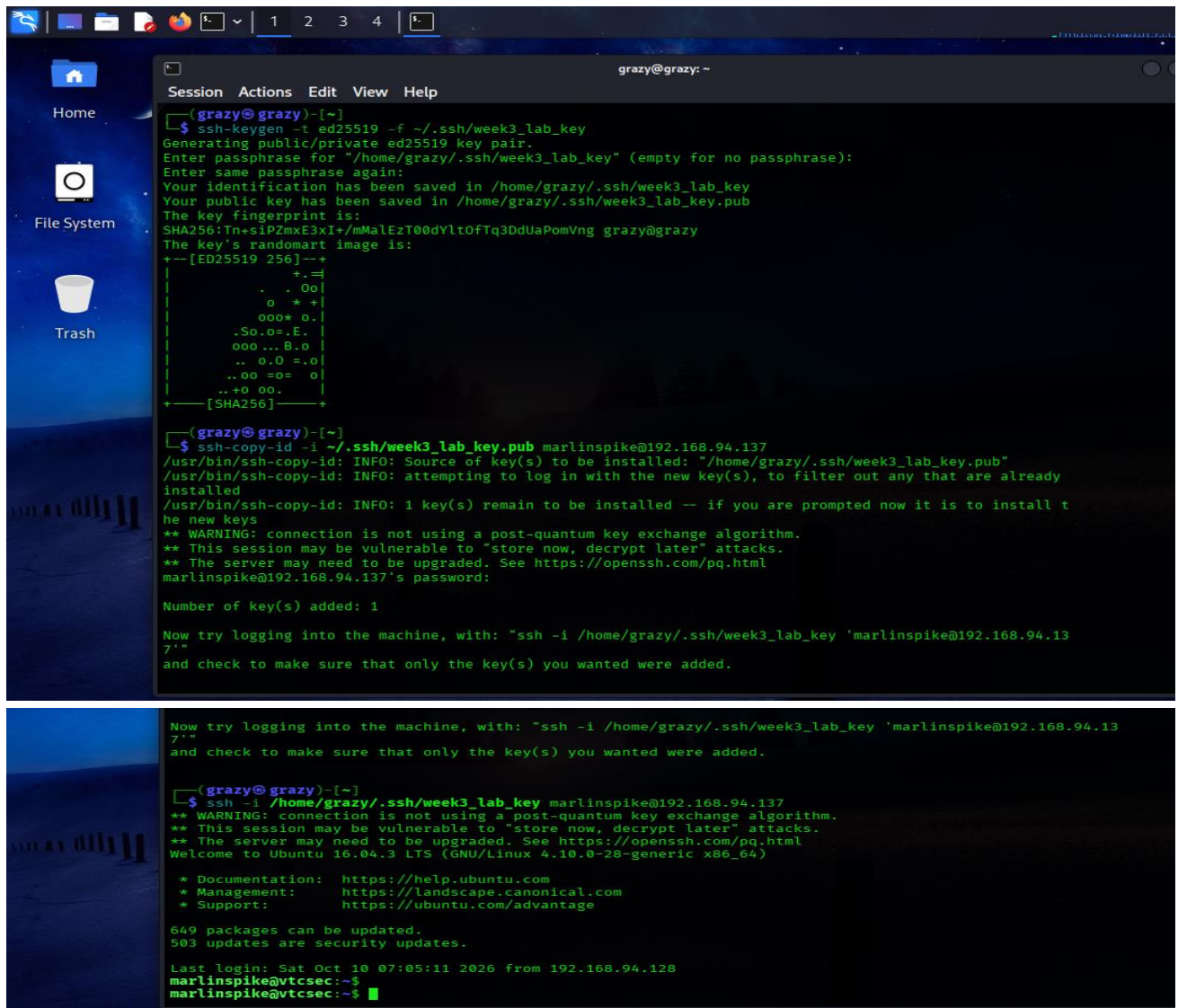
Practical Observation:

The results confirmed that the SSH session was connected to the intended target system and that the correct user account was being used.

26. SSH Key Authentication

SSH supports key-based authentication as an alternative to password authentication. This method uses a pair of cryptographic keys: a **private key** that remains on the client computer and a **public key** that is placed on the SSH server.

Key-based authentication can provide stronger protection than relying only on passwords when it is properly configured and managed. It is commonly used to provide secure access to remote Linux systems while reducing reliance on password-based authentication.



```
(grazy@grazy)~$ ssh-keygen -t ed25519 -f ~/.ssh/week3_lab_key
Generating public/private ed25519 key pair.
Enter passphrase for "/home/grazy/.ssh/week3_lab_key" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/grazy/.ssh/week3_lab_key
Your public key has been saved in /home/grazy/.ssh/week3_lab_key.pub
The key fingerprint is:
SHA256:Tn+siPZmxE3xI+/mMalEzT00dYltOfTq3DdUaPomVng grazy@grazy
The key's randomart image is:
+--[ED25519 256]--+
      .+.=
      . .Oo|
      o  * +|
      ooo* o.|
      .So.o=.E.|
      ooo...B.o|
      .. o.O =.o|
      ..oo =o= o|
      ..+o oo.|
+-----[SHA256]-----

(grazy@grazy)~$ ssh-copy-id -i ~/.ssh/week3_lab_key.pub marlinspike@192.168.94.137
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/grazy/.ssh/week3_lab_key.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already
installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install t
he new keys
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
marlinspike@192.168.94.137's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/grazy/.ssh/week3_lab_key 'marlinspike@192.168.94.13
7'"
and check to make sure that only the key(s) you wanted were added.

(grazy@grazy)~$ ssh -i /home/grazy/.ssh/week3_lab_key marlinspike@192.168.94.137
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

649 packages can be updated.
503 updates are security updates.

Last login: Sat Oct 10 07:05:11 2026 from 192.168.94.128
marlinspike@vtcsec:~$
marlinspike@vtcsec:~$
```

Practical Observation:

SSH key authentication provides an additional method of securely authenticating users when accessing remote Linux systems.

27. SSH Security Hardening

SSH provides remote access to a Linux system, so it must be configured carefully to reduce the possibility of unauthorized access. Several security practices can be applied to improve SSH security.

- **Avoid Unnecessary Root Login**

Direct remote access using the root account should generally be restricted because compromising a root account could provide an attacker with extensive control over the system.

- **Use Strong Authentication**

Strong passwords and properly managed SSH keys can help reduce the risk of unauthorized access.

- **Use Dedicated Accounts**

Dedicated user accounts should be used instead of unnecessarily using highly privileged accounts. This helps limit the amount of access available to each user.

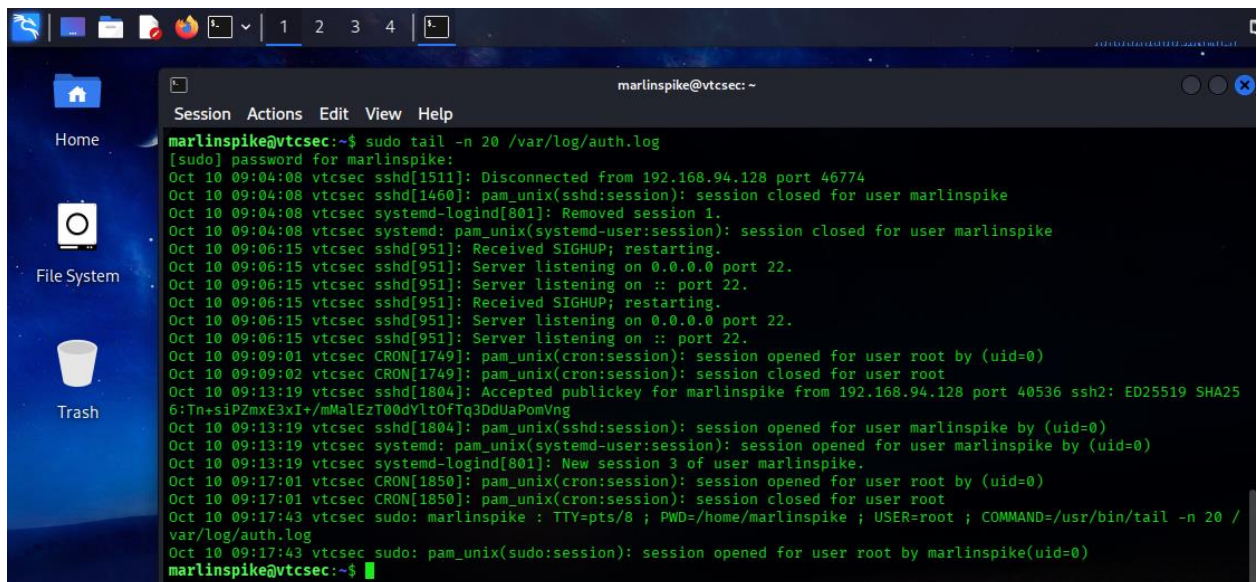
- **Keep SSH Software Updated**

Keeping the operating system and SSH software updated helps address known security vulnerabilities.

- **Monitor Authentication Activity**

SSH authentication activity can be monitored through system logs. Monitoring these logs can help administrators identify successful and unsuccessful login attempts and detect suspicious activity.

SSH authentication activity in the system logs

A screenshot of a Linux desktop environment. On the left, there is a sidebar with icons for Home, File System, and Trash. The main area is a terminal window titled 'marlinspike@vtcsec: ~'. The terminal shows the command 'sudo tail -n 20 /var/log/auth.log' being executed. The output of the command displays various system logs, including SSH disconnections, session closures, and successful logins. Key entries include 'Accepted publickey for marlinspike from 192.168.94.128 port 40536 ssh2: ED25519 SHA256:Tn+siPZmxE3xI+/mMalEzT00dYltOfTq3DdUaPomVng' and 'session opened for user marlinspike by (uid=0)'. The terminal prompt is 'marlinspike@vtcsec:~\$'.

The authentication log shows **Accepted publickey for marlinspike from 192.168.94.128**, confirming successful key-based login using the ED25519 key generated earlier. The log also shows **Server listening on 0.0.0.0 port 22**, indicating the SSH daemon is active.

Practical Observation:

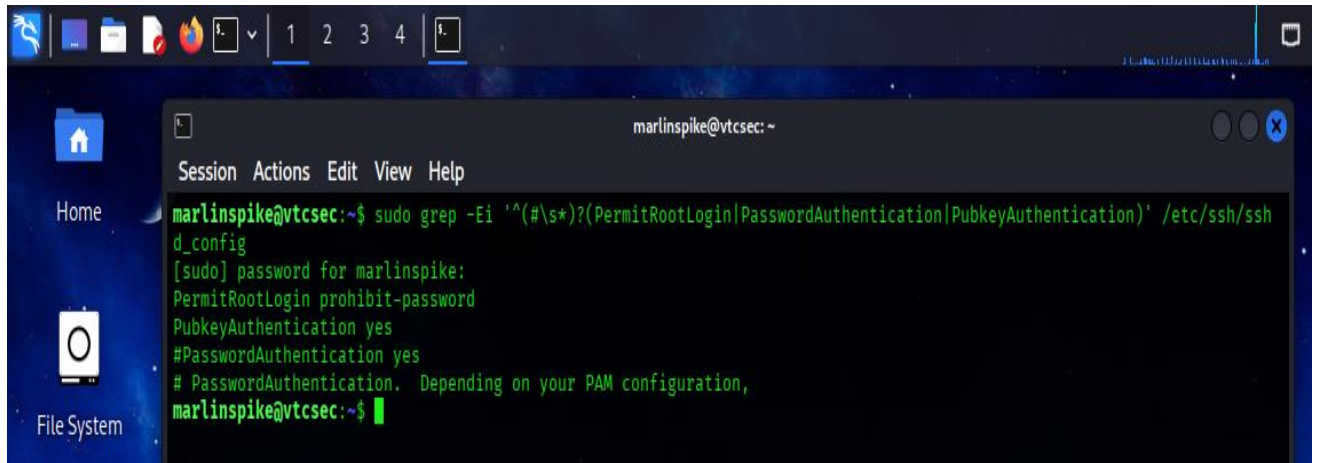
Proper SSH hardening can reduce the risk of unauthorized remote access and improve the security of Linux systems.

28. SSH Configuration File

The SSH server uses a configuration file to control important aspects of its operation. The configuration includes settings related to authentication and remote access.

Important SSH security settings include root login, password authentication, and public-key authentication. These settings should be configured carefully because an incorrect configuration could prevent legitimate users from accessing the server.

SSH server configuration file



The filtered output displays the active security directives. PermitRootLogin prohibit-password secures the root account against password brute-force, while PubkeyAuthentication yes confirms that public-key authentication is enabled, supporting the successful key login shown in Figure 27.

Practical Observation:

The SSH configuration file provides administrators with control over important authentication and access settings.

29. Practical Workflow

The practical exercise followed a structured process beginning with the preparation of the virtual machines and ending with verification of the remote SSH connection.

First, Kali Linux and Basic Pentesting 1 were started and checked. The IP address of the Ubuntu-based system was identified, and both machines were confirmed to be connected to the same VMware virtual network.

The SSH service on the target system was then checked to confirm that it was operational. Network connectivity between the two machines was tested before establishing the SSH connection.

After successful connectivity was confirmed, Kali Linux was used to establish an SSH connection to the target system. The remote session was then verified by checking the logged-in user and hostname.

Finally, SSH security concepts such as key-based authentication, strong authentication, dedicated accounts, restricted administrative access, software updates, and authentication monitoring were reviewed.

30. Practical Results

The practical exercise provided experience in several areas of Linux security.

Practical Area	Result
Linux command-line navigation	practiced
File creation	practiced
File permissions	practiced
File permission modification	practiced
User identification	practiced
User and group management	practiced
Ubuntu IP identification	practiced
SSH service management	practiced
Network connectivity	tested
Kali to Ubuntu SSH connection	Successfully established
SSH authentication	Successfully performed
SSH security concepts	studied
SSH key authentication	

31. Security Lessons Learned

This task demonstrated that Linux security depends on several security controls working together.

File permissions control who can access and modify files, while user management controls who can access the system and what privileges they have. SSH provides a secure method for remotely accessing Linux systems.

The practical also demonstrated the importance of authentication because the system needs to verify the identity of users before allowing access. In addition, security configuration and monitoring are important because simply running SSH does not automatically make a system completely secure.

Practical Observation:

The practical demonstrated that effective Linux security requires proper management of users, permissions, authentication, services, and access controls.

32. Challenges Encountered

One of the challenges encountered during the preparation of this task was the Ubuntu virtual machine installation. The original Ubuntu installation experienced problems, making it difficult to use for the practical.

To continue the practical, an existing Ubuntu-based virtual machine called Basic Pentesting 1 was selected. This allowed the required Linux and SSH concepts to be practiced in a controlled environment while keeping Kali Linux as the main cybersecurity workstation.

Another challenge was ensuring that both virtual machines were connected to the same VMware network so that they could communicate using the 192.168.94.x address range.

33. Conclusion

The Week 3 Linux and Command Line Security task provided practical experience with important Linux security concepts. The task covered Linux command-line operations, file permissions, user management, and Secure Shell (SSH).

The file-permission activities demonstrated how Linux controls access to files through read, write, and execute permissions. The user-management activities demonstrated how accounts and groups can be used to control access to system resources.

The SSH practical demonstrated how Kali Linux can securely connect to an Ubuntu system through a virtual network. Kali Linux was used as the main cybersecurity workstation and SSH client, while Basic Pentesting 1 was used as the Ubuntu SSH server.

The practical also introduced important SSH security measures, including strong authentication, dedicated user accounts, key-based authentication, restricted administrative access, software updates, and authentication-log monitoring.

Overall, the task improved my understanding of Linux command-line operations and demonstrated how Linux security controls can be applied in a practical cybersecurity environment.